

CollabServe AI

Full-Stack Application Assessment Documentation

Detailed architecture, folder structure, validation mapping, integration flow, runtime behavior, and reviewer traceability for D:\FinalProject.

Generated artifact: CollabServe_AI_Project_Documentation.pdf

This document is technical documentation and not a substitute for any required authorship disclosure.

2. Rubric Compliance Matrix

Parameter	Implementation location	Detailed evidence
Authentication & User Flow	Login.jsx, Register.jsx, UserContext.jsx, App.jsx	Contexted tokens are authenticated to backend routes. UserContext rehydrates sessions from /users/profile
Project & Task Management UI	Home.jsx, Project.jsx	Dashboard creates, searches, summarizes, and opens projects. Workspace manages chat, members
Dashboard & Data Presentation	Home.jsx	Metric cards show project count, collaborator memberships, and saved files. Project cards show file c
Validations, Error & Loading States	Routes, controllers, services, Home.jsx, Project.jsx	Express-validator checks inputs. Services enforce ObjectIds, membership, ownership, duplicate nam
Code Quality & Responsiveness	Backend service split and frontend screen layout	Route/controller/service/model boundaries are clear. Derived React values use memoization. Deskto
REST API Design & Coverage	backend/routes/*.js	Routes cover health, auth, profile, logout, user list, project create/list/read, collaborator update, file tre
Authentication & Security	user.model.js, auth.middleware.js, redis.service.js, api.middleware.js	Backend-only AI keys, and API keys, and password exclusion, env-based CORS, backend-only AI keys, and API
Role-Based Access Control	project.model.js, project.service.js, Project.jsx	Project owner is separate from member list. Members can access a project; only the owner can add
Database Design & Relationships	user.model.js, project.model.js	Users and projects are separate Mongo documents. Projects reference owner and members by Objec
Visual Quality	Home.jsx, Project.jsx, Login.jsx, Register.jsx, Workspace.jsx	Consistent surfaces, readable type, icon controls, status feedback, practical spacing, responsive sec

3. Actual Folder Structure

Generated/dependency folders such as .git, node_modules, dist, logs, and the temporary PDF generator are excluded from this view.

```
FinalProject/
.env.example
.gitignore
CollabServe_AI_Project_Documentation.pdf
README.md
package-lock.json
package.json
backend/
  .env.example
  app.js
  body.json
  package-lock.json
  package.json
  server.js
  controllers/
    project.controller.js
    user.controller.js
  db/
  db.js
  middleware/
    auth.middleware.js
  models/
    project.model.js
    user.model.js
  routes/
    ai.routes.js
    project.routes.js
    user.routes.js
  services/
    ai.service.js
    project.service.js
    redis.service.js
    user.service.js
frontend/
  .env.example
  eslint.config.js
  index.html
  package-lock.json
  package.json
  postcss.config.js
  tailwind.config.js
  vite.config.js
  src/
    App.jsx
    index.css
    main.jsx
    config/
      axios.js
      socket.js
      webcontainer.js
    context/
      UserContext.jsx
    routes/
      AppRoutes.jsx
    screens/
      Home.jsx
      Login.jsx
      Project.jsx
      Register.jsx
```

4. File-by-File Responsibility Map

File	Responsibility
backend/app.js	Express composition: CORS, middleware, health route, route mounting, and shared JSON error shape.
backend/server.js	Mongo boot, HTTP server, Socket.IO authentication, room join, chat broadcast, and @ai handling.
backend/db/db.js	MongoDB connection helper requiring MONGODB_URI.
backend/models/user.model.js	User schema, email rules, password hashing, password check, and JWT generation.
backend/models/project.model.js	Project schema with owner, users, fileTree, timestamps, and owner/name unique index.
backend/middleware/auth.middleware.js	Token extraction, blacklist check, JWT verification, safe user loading, and req.user attachment.
backend/routes/user.routes.js	Register, login, profile, logout, and user list endpoints with validation.
backend/routes/project.routes.js	Project and file endpoints with ObjectId and payload validation.
backend/routes/ai.routes.js	Authenticated prompt endpoint for Gemini results.
backend/controllers/user.controller.js	User request/response handling and safe output shaping.
backend/controllers/project.controller.js	Project request/response handling and service error mapping.
backend/services/user.service.js	Email normalization, duplicate prevention, password hashing, and safe user listing.
backend/services/project.service.js	Create/list/read/update file tree plus membership and owner-only collaborator business logic.
backend/services/redis.service.js	Optional Redis logout blacklist with memory fallback.
backend/services/ai.service.js	Gemini wrapper, strict JSON contract, cleanup, validation, and key redaction.
frontend/src/App.jsx	Top-level UserProvider and route wrapper.
frontend/src/routes/AppRoutes.jsx	Public and protected routing.
frontend/src/context/UserContext.jsx	Session bootstrapping, localStorage synchronization, and context helper.
frontend/src/config/axios.js	API base URL and bearer token injection.
frontend/src/config/socket.js	Socket client initialize/receive/send/disconnect helpers.
frontend/src/config/webcontainer.js	Singleton WebContainer boot helper.
frontend/src/screens/Login.jsx	Login form, API request, loading/error UI, token persistence, navigation.
frontend/src/screens/Register.jsx	Registration form, API request, loading/error UI, token persistence, navigation.
frontend/src/screens/Home.jsx	Dashboard metrics, search, project creation modal, loading/error/empty states, logout.
frontend/src/screens/Project.jsx	Workspace chat, members, files, editor, AI file merge, WebContainer run, terminal, preview, responsive layout.
frontend/src/index.css	Tailwind layers, base defaults, inherited form fonts, and scrollbar styling.

1. Project Purpose

CollabServe AI is a collaborative coding workspace. It combines identity, project management, realtime communication, AI generation, persistent file storage, browser execution, terminal output, and preview in one full-stack flow.

The goal is to satisfy the assessment categories from the screenshot with real behavior: user flow, project management, dashboard data, validation, backend API coverage, security, RBAC, database relationships, and polished responsive UI.

5. Backend Architecture

The backend is organized as route, controller, service, model, middleware, and server layers. This keeps HTTP details out of business logic and keeps data rules out of the UI.

app.js owns the Express pipeline. server.js owns database startup and realtime socket authorization. Controllers translate requests into service calls. Services enforce domain rules. Models define MongoDB data shape and reusable document methods.

The socket middleware mirrors REST authorization by checking JWT, project id validity, user existence, project existence, and project membership before the socket can join a room.

6. Database Design

The User collection stores identity. Email is unique, lowercased, trimmed, and length-limited. Password is stored as a bcrypt hash and hidden from normal queries.

The Project collection stores owner, members, and fileTree. Owner is the elevated role. Users is the membership list. fileTree is embedded because it is project-owned workspace state that is loaded and saved together.

A compound index on owner and name prevents duplicate project names per owner while allowing different users to choose the same name independently.

7. Frontend Architecture

The frontend uses UserContext for global session state and screen-local state for dashboard and workspace interactions. This avoids pushing temporary UI state into global context.

Home.jsx is a data dashboard. It loads projects, computes metrics, filters by search, creates projects, shows loading/error/empty states, and handles logout.

Project.jsx is the main workbench. It loads project data and users, opens files, manages chat, controls collaborator selection, receives AI file output, saves fileTree changes, boots WebContainer, runs code, streams terminal output, and previews server URLs.

8. Integration Flow

Login begins in Login.jsx, posts to /users/login, passes through validator and controller logic, compares bcrypt password, receives JWT, saves token, updates UserContext, and opens the dashboard.

Project creation begins in Home.jsx, posts to /projects/create, passes through auth middleware, creates a MongoDB project with owner and members, then prepends the returned project into dashboard state.

AI generation begins with an @ai chat message, travels through Socket.IO, calls Gemini through ai.service.js, validates strict JSON, emits the AI response, merges fileTree into Project.jsx state, persists it through /projects/update-file-tree, and opens the generated file.

User -> React screen -> Axios/Socket helper -> Express/Socket middleware -> Controller/Service -> MongoDB/Gemini -> JSON r

9. Validation Strategy

Validation is layered. The browser catches obvious mistakes for user experience. express-validator rejects malformed input. Services enforce business rules. Mongoose enforces data shape. This matters because frontend checks can be bypassed.

Important validations include email format, password length, required project name, valid ObjectIds, users array shape, fileTree object shape, project membership, owner-only collaborator changes, non-empty AI prompt, and valid AI JSON.

10. Visual and UX Quality

The dashboard uses a dark, high-contrast operational style. It avoids a decorative landing page and starts directly with useful project data. Cards are compact and scannable.

The project workspace is utility-focused. Chat, files, editor, preview, terminal, and member management are separate zones. On desktop they form a productive grid; on smaller screens they stack instead of forcing horizontal scrolling.

Loading states, empty states, disabled states, icons, and error surfaces are intentionally visible so users understand what the system is doing.

11. Security Review

Secrets belong only in backend environment files. Gemini keys are never exposed through Vite variables. CORS follows CLIENT_URL instead of wildcard origin. JWTs are verified on REST and socket flows.

The backend, not the UI, enforces owner-only collaborator management. UI disabled states are helpful but not trusted as the security boundary.

12. Setup and Operation

Install dependencies with `npm install` and `npm run install:all`. Copy `backend/.env.example` to `backend/.env` and `frontend/.env.example` to `frontend/.env`. Configure MongoDB, JWT_SECRET, CLIENT_URL, VITE_API_URL, and GEMINI_API_KEY.

Run `npm run dev` from the root. The frontend normally runs on `http://localhost:5173` and the backend normally runs on `http://localhost:3000`.

```
npm install
npm run install:all
copy backend\.env.example backend\.env
copy frontend\.env.example frontend\.env
npm run dev

Frontend: http://localhost:5173
Backend: http://localhost:3000
```

13. Verification Evidence

`npm.cmd run lint` completed successfully. `npm.cmd run build` completed successfully after Vite/esbuild was allowed to spawn outside the sandbox. `node --check` passed for `backend/server.js`, `backend/services/ai.service.js`, and `backend/routes/project.routes.js`.

Vite reported a large chunk warning because browser execution tooling is heavy. That warning does not indicate build failure.

14. Maintenance Notes

For demo, register two users, create a project as the first user, open it, add the second user as collaborator, send a normal chat message, send an @ai prompt, inspect generated files, edit a file, run the project, and show preview plus terminal output.

If AI fails, check GEMINI_API_KEY and GEMINI_MODEL. If sockets fail, check token, backend URL, and project membership. If preview fails, check package.json scripts or selected JavaScript file.

15. Integrity Note

This document explains structure and implementation decisions. It should be used for review, maintenance, and study. Any school or assessment rule requiring disclosure of tool assistance should be followed honestly.

16. REST API Reference

Area	Endpoint	Method	Input	Behavior
Health	/	GET	None	Returns API status and capability checks.
Auth	/users/register	POST	email, password	Creates user, hashes password, returns safe user and JWT.
Auth	/users/login	POST	email, password	Verifies password and returns safe user and JWT.
Auth	/users/profile	GET	Bearer token	Returns the authenticated user from middleware.
Auth	/users/logout	GET	Bearer token	Blacklists the token for 24 hours.
Users	/users/all	GET	Bearer token	Returns possible collaborators excluding current user.
Projects	/projects/create	POST	name	Creates project with owner and initial users array.

Area	Endpoint	Method	Input	Behavior
Projects	/projects/all	GET	Bearer token	Lists projects where current user is a member.
Projects	/projects/add-user	PUT	projectId, users[]	Owner-only collaborator add-to-set update.
Projects	/projects/get-project/:projectId	GET	projectId	Loads populated project only for members.
Projects	/projects/update-file-tree	PUT	projectId, fileTree	Persists editor/AI file tree state.
AI	/ai/get-result	POST	prompt	Returns strict JSON string generated by Gemini.

17. Reviewer Traceability Summary

- Authentication is traceable from Login/Register to user routes, controllers, user model methods, JWT return, localStorage, and protected route hydration.
- Project creation is traceable from Home.jsx to project routes, auth middleware, project service, project model, MongoDB persistence, and dashboard state update.
- Collaboration is traceable from Project.jsx controls to owner-only project service rules and populated member display.
- AI generation is traceable from @ai chat message to Socket.IO, Gemini service, strict JSON validation, fileTree merge, persistence, and editor display.
- Running code is traceable from the Run button to WebContainer mount, dependency install, process spawn, output streaming, server-ready URL, and iframe preview.

Final position: the project now directly addresses the screenshot parameters through implemented behavior across frontend, backend, database, realtime events, AI integration, documentation, and environment hygiene.